

Week 05: Geometry Calculator — Blazor Web App

Step-by-Step Guide (Visual Studio 2022)

Part 1: Create the Project

Step 1: Create the Blazor Web App

1. Open **Visual Studio 2022**
2. Click **Create a new project**
3. Select **Blazor Web App** → Click **Next**
4. Project name: GeometryCalculatorBlazor → Choose location → Click **Next**
5. Configure:
 - Framework: **.NET 8.0**
 - Authentication: **None**
 - Interactive render mode: **Server**
 - Interactivity location: **Global**
 - ☐ Do not use top-level statements (leave unchecked)
6. Click **Create**

Note: Selecting **Server** interactive render mode with **Global** interactivity means all pages will be interactive by default — buttons and events will work everywhere without extra configuration.

Part 2: Create the Models

Step 2: Create the Models Folder

Right-click the project → **Add** → **New Folder** → Name it Models.

2a. Create ShapeRequest.cs

Right-click Models → **Add** → **Class** → Name: ShapeRequest.cs

```
namespace GeometryCalculatorBlazor.Models
{
    public class ShapeRequest
    {
        public string Shape { get; set; } = string.Empty;
        public Dictionary<string, double> Properties { get; set; } = new();
    }
}
```

2b. Create ShapeResult.cs

Right-click Models → **Add** → **Class** → Name: ShapeResult.cs

```
namespace GeometryCalculatorBlazor.Models
{
    public class ShapeResult
    {
        public string Shape { get; set; } = string.Empty;
        public double Area { get; set; }
        public double Circumference { get; set; }
        public Dictionary<string, double> InputProperties { get; set; } =
new();
    }
}
```

Part 3: Create the Calculation Service

Step 3: Create the Services Folder

Right-click the project → **Add** → **New Folder** → Name it Services.

3a. Create IGeometryService.cs

Right-click Services → **Add** → **New Item** → **Interface** → Name: IGeometryService.cs

```
using GeometryCalculatorBlazor.Models;

namespace GeometryCalculatorBlazor.Services
{
    public interface IGeometryService
    {
        ShapeResult Calculate(ShapeRequest request);
        List<string> GetSupportedShapes();
        List<string> GetShapeProperties(string shapeName);
    }
}
```

What changed? We added GetShapeProperties() to the interface. In the Web API version, the controller held the properties map. In Blazor, the service owns all geometry logic — the Razor component simply calls the service. This keeps our UI layer thin and testable.

3b. Create GeometryService.cs

Right-click Services → **Add** → **Class** → Name: GeometryService.cs

```
using GeometryCalculatorBlazor.Models;
```

```

namespace GeometryCalculatorBlazor.Services
{
    public class GeometryService : IGeometryService
    {
        // Properties map – defines what inputs each shape requires
        private readonly Dictionary<string, List<string>> _propertiesMap
            = new(StringComparer.OrdinalIgnoreCase)
        {
            { "Circle",          new List<string> { "radius" } },
            { "Rectangle",       new List<string> { "length", "width" } },
            { "Triangle",        new List<string> { "sideA", "sideB", "sideC" } },
            { "Square",          new List<string> { "side" } },
            { "Ellipse",         new List<string> { "semiMajorAxis",
"semiMinorAxis" } },
            { "Parallelogram",    new List<string> { "base", "height", "side" } },
            { "Trapezoid",       new List<string> { "topBase", "bottomBase",
"height", "leftSide",
"rightSide" } }
        };

        public List<string> GetSupportedShapes()
        {
            return _propertiesMap.Keys.ToList();
        }

        public List<string> GetShapeProperties(string shapeName)
        {
            if (_propertiesMap.TryGetValue(shapeName, out var properties))
                return properties;

            throw new ArgumentException($"Shape '{shapeName}' is not
supported.");
        }

        public ShapeResult Calculate(ShapeRequest request)
        {
            var result = new ShapeResult
            {
                Shape = request.Shape,
                InputProperties = request.Properties
            };

            switch (request.Shape.ToLower())
            {
                case "circle":
                    CalculateCircle(request.Properties, result);
                    break;
            }
        }
    }
}

```

```

        case "rectangle":
            CalculateRectangle(request.Properties, result);
            break;
        case "triangle":
            CalculateTriangle(request.Properties, result);
            break;
        case "square":
            CalculateSquare(request.Properties, result);
            break;
        case "ellipse":
            CalculateEllipse(request.Properties, result);
            break;
        case "parallelogram":
            CalculateParallelogram(request.Properties, result);
            break;
        case "trapezoid":
            CalculateTrapezoid(request.Properties, result);
            break;
        default:
            throw new ArgumentException($"Unsupported shape:
{request.Shape}");
    }

    // Round to 4 decimal places
    result.Area = Math.Round(result.Area, 4);
    result.Circumference = Math.Round(result.Circumference, 4);

    return result;
}

private void CalculateCircle(Dictionary<string, double> props,
ShapeResult result)
{
    double radius = GetProperty(props, "radius");
    result.Area = Math.PI * radius * radius;
    result.Circumference = 2 * Math.PI * radius;
}

private void CalculateRectangle(Dictionary<string, double> props,
ShapeResult result)
{
    double length = GetProperty(props, "length");
    double width = GetProperty(props, "width");
    result.Area = length * width;
    result.Circumference = 2 * (length + width);
}

private void CalculateTriangle(Dictionary<string, double> props,
ShapeResult result)

```

```

    {
        double sideA = GetProperty(props, "sideA");
        double sideB = GetProperty(props, "sideB");
        double sideC = GetProperty(props, "sideC");

        // Heron's formula for area
        double s = (sideA + sideB + sideC) / 2;
        result.Area = Math.Sqrt(s * (s - sideA) * (s - sideB) * (s -
sideC));
        result.Circumference = sideA + sideB + sideC;
    }

    private void CalculateSquare(Dictionary<string, double> props,
ShapeResult result)
    {
        double side = GetProperty(props, "side");
        result.Area = side * side;
        result.Circumference = 4 * side;
    }

    private void CalculateEllipse(Dictionary<string, double> props,
ShapeResult result)
    {
        double semiMajor = GetProperty(props, "semiMajorAxis");
        double semiMinor = GetProperty(props, "semiMinorAxis");
        result.Area = Math.PI * semiMajor * semiMinor;
        // Ramanujan's approximation for circumference
        double h = Math.Pow(semiMajor - semiMinor, 2) /
            Math.Pow(semiMajor + semiMinor, 2);
        result.Circumference = Math.PI * (semiMajor + semiMinor) *
            (1 + (3 * h) / (10 + Math.Sqrt(4 - 3 *
h))));
    }

    private void CalculateParallelogram(Dictionary<string, double> props,
ShapeResult result)
    {
        double baseLength = GetProperty(props, "base");
        double height = GetProperty(props, "height");
        double side = GetProperty(props, "side");
        result.Area = baseLength * height;
        result.Circumference = 2 * (baseLength + side);
    }

    private void CalculateTrapezoid(Dictionary<string, double> props,
ShapeResult result)
    {
        double topBase = GetProperty(props, "topBase");
        double bottomBase = GetProperty(props, "bottomBase");

```

```

        double height = GetProperty(props, "height");
        double leftSide = GetProperty(props, "leftSide");
        double rightSide = GetProperty(props, "rightSide");
        result.Area = 0.5 * (topBase + bottomBase) * height;
        result.Circumference = topBase + bottomBase + leftSide +
rightSide;
    }

    private double GetProperty(Dictionary<string, double> props, string
key)
    {
        if (!props.ContainsKey(key))
            throw new ArgumentException($"Missing required property:
{key}");

        if (props[key] <= 0)
            throw new ArgumentException($"Property '{key}' must be
greater than zero.");

        return props[key];
    }
}

```

Part 4: Register the Service

Step 4: Update Program.cs

Open Program.cs and add the service registration. Insert the highlighted line:

```

using GeometryCalculatorBlazor.Components;
using GeometryCalculatorBlazor.Services;

var builder = WebApplication.CreateBuilder(args);

// Add services to the container
builder.Services.AddRazorComponents()
    .AddInteractiveServerComponents();

// Register the Geometry Service with Dependency Injection
builder.Services.AddScoped<IGeometryService, GeometryService>();

var app = builder.Build();

// Configure the HTTP request pipeline
if (!app.Environment.IsDevelopment())
{
    app.UseExceptionHandler("/Error", createScopeForErrors: true);
}

```

```

        app.UseHsts();
    }

    app.UseHttpsRedirection();
    app.UseStaticFiles();
    app.UseAntiforgery();

    app.MapRazorComponents<App>()
        .AddInteractiveServerRenderMode();

    app.Run();

```

Key Difference from Web API: There are no controllers, no Swagger, no CORS. Blazor renders the UI on the server and communicates via SignalR — the service is called directly in C# from the Razor component, with no HTTP requests involved.

Part 5: Create the Blazor Component (the UI)

Step 5: Replace the Home Page

In the Blazor Web App template, the home page is located at:

Components/Pages/Home.razor

Open Components/Pages/Home.razor and **replace its entire content** with the following:

```

@page "/"
@using GeometryCalculatorBlazor.Models
@using GeometryCalculatorBlazor.Services
@inject IGeometryService GeometryService

<PageTitle>Geometry Calculator</PageTitle>

<div class="calculator-card">
    <!-- Header -->
    <div class="card-header">
        <h1>📐 Geometry Calculator</h1>
        <p>Calculate area & circumference of shapes</p>
    </div>

    <div class="card-body">
        <!-- Step 1: Select Shape -->
        <div class="form-group">
            <label>1. Select the Shape</label>
            <select @onchange="OnShapeSelected">
                <option value="">-- Choose a shape --</option>
                @foreach (var shape in supportedShapes)

```

```

        {
            <option value="@shape">@shape</option>
        }
    </select>
</div>

<!-- Step 2: Enter Properties (dynamic) -->
@if (shapeProperties.Count > 0)
{
    <div class="form-group">
        <label>2. Enter the Shape Properties</label>
        @foreach (var prop in shapeProperties)
        {
            <div class="property-input">
                <label>@FriendlyLabel(prop)</label>
                <input type="number"
                    placeholder="@($"Enter
{FriendlyLabel(prop).ToLower()}")"
                    min="0.01"
                    step="any"
                    @oninput="(e) => OnPropertyChanged(prop, e)"
                />
            </div>
        }
    </div>
}

<!-- Step 3: Calculate -->
<button class="btn-calculate"
    disabled="@(!canCalculate)"
    @onclick="Calculate">
    Calculate
</button>

<!-- Error Message -->
@if (!string.IsNullOrEmpty(errorMessage))
{
    <div class="error-message visible">@errorMessage</div>
}

<!-- Results -->
@if (result != null)
{
    <div class="results-panel visible">
        <h3>Results for @result.Shape</h3>
        <div class="result-row">
            <span class="result-label">Area</span>
            <span class="result-value">@result.Area.ToString("N4")
sq. units</span>
        </div>
    </div>
}

```



```

        </div>
        <div class="result-row">
            <span class="result-label">Circumference /
Perimeter</span>
            <span class="result-
value">@result.Circumference.ToString("N4") units</span>
        </div>
    </div>
}
</div>
</div>

@code {
    // ===== State Fields =====
    private List<string> supportedShapes = new();
    private List<string> shapeProperties = new();
    private Dictionary<string, double> propertyValues = new();
    private string selectedShape = string.Empty;
    private string? errorMessage;
    private ShapeResult? result;
    private bool canCalculate => !string.IsNullOrEmpty(selectedShape)
                                && shapeProperties.Count > 0;

    // ===== Lifecycle: Load shapes when the component initialises =====
    protected override void OnInitialized()
    {
        supportedShapes = GeometryService.GetSupportedShapes();
    }

    // ===== Event: Shape dropdown changed =====
    private void OnShapeSelected(ChangeEventArgs e)
    {
        selectedShape = e.Value?.ToString() ?? string.Empty;
        result = null;
        errorMessage = null;
        propertyValues.Clear();

        if (string.IsNullOrEmpty(selectedShape))
        {
            shapeProperties.Clear();
            return;
        }

        shapeProperties = GeometryService.GetShapeProperties(selectedShape);
    }

    // ===== Event: A property input value changed =====
    private void OnPropertyChanged(string propertyName, ChangeEventArgs e)
    {

```

```

        if (double.TryParse(e.Value?.ToString(), out double value))
        {
            propertyValues[propertyName] = value;
        }
        else
        {
            propertyValues.Remove(propertyName);
        }
    }

// ===== Event: Calculate button clicked =====
private void Calculate()
{
    errorMessage = null;
    result = null;

    // Validate all properties have values
    foreach (var prop in shapeProperties)
    {
        if (!propertyValues.ContainsKey(prop) || propertyValues[prop] <=
0)
        {
            errorMessage = "Please enter valid positive numbers for all
properties.";
            return;
        }
    }

    try
    {
        var request = new ShapeRequest
        {
            Shape = selectedShape,
            Properties = new Dictionary<string, double>(propertyValues)
        };

        result = GeometryService.Calculate(request);
    }
    catch (ArgumentException ex)
    {
        errorMessage = ex.Message;
    }
}

// ===== Helper: Convert property keys to friendly display labels =====
private string FriendlyLabel(string key) => key switch
{
    "radius"          => "Radius",
    "length"          => "Length",

```

```

        "width"           => "Width",
        "side"            => "Side",
        "sideA"           => "Side A",
        "sideB"           => "Side B",
        "sideC"           => "Side C",
        "semiMajorAxis"   => "Semi-Major Axis",
        "semiMinorAxis"   => "Semi-Minor Axis",
        "base"            => "Base",
        "height"          => "Height",
        "topBase"         => "Top Base",
        "bottomBase"      => "Bottom Base",
        "leftSide"        => "Left Side",
        "rightSide"       => "Right Side",
        -                 => key
    };
}

```

What's happening here? Instead of JavaScript `fetch()` calls to an API, we use `@inject` to get the `GeometryService` directly. Shape selection, property input, and calculation are all handled by C# methods in the `@code` block. Blazor's data binding (`@onchange`, `@oninput`, `@onclick`) replaces all the DOM manipulation that was done in JavaScript.

Part 6: Add the Styling

Step 6: Create the Component CSS File

Blazor supports **CSS isolation** — each component can have its own scoped stylesheet.

Create a new file alongside the Razor component:

`Components/Pages/Home.razor.css`

Right-click `Components/Pages` → **Add** → **New Item** → **Style Sheet** → Name: `Home.razor.css`

Paste the following:

```

/* ===== Reset & Base ===== */
::deep *, ::deep *::before, ::deep *::after {
    box-sizing: border-box;
    margin: 0;
    padding: 0;
}

/* ===== Card Container ===== */
.calculator-card {
    background: #fff;
    border-radius: 16px;
}

```

```

    box-shadow: 0 20px 60px rgba(0, 0, 0, 0.3);
    width: 100%;
    max-width: 520px;
    overflow: hidden;
    margin: 40px auto;
}

.card-header {
    background: linear-gradient(135deg, #667eea, #764ba2);
    color: #fff;
    padding: 28px 32px;
    text-align: center;
}

.card-header h1 {
    font-size: 1.6rem;
    font-weight: 600;
    margin-bottom: 4px;
}

.card-header p {
    font-size: 0.9rem;
    opacity: 0.85;
}

.card-body {
    padding: 32px;
}

/* ===== Form Elements ===== */
.form-group {
    margin-bottom: 20px;
}

.form-group > label {
    display: block;
    font-weight: 600;
    margin-bottom: 8px;
    font-size: 0.9rem;
    color: #444;
}

select, input[type="number"] {
    width: 100%;
    padding: 12px 16px;
    border: 2px solid #e0e0e0;
    border-radius: 8px;
    font-size: 1rem;
    transition: border-color 0.2s;
}

```

```

    outline: none;
}

select:focus, input[type="number"]:focus {
    border-color: #667eea;
}

input[type="number"]::placeholder {
    color: #aaa;
}

/* ===== Properties Container ===== */
.property-input {
    display: flex;
    align-items: center;
    gap: 12px;
    margin-bottom: 12px;
}

.property-input label {
    min-width: 130px;
    font-weight: 500;
    font-size: 0.88rem;
    color: #555;
    text-transform: capitalize;
}

.property-input input {
    flex: 1;
}

/* ===== Button ===== */
.btn-calculate {
    width: 100%;
    padding: 14px;
    background: linear-gradient(135deg, #667eea, #764ba2);
    color: #fff;
    border: none;
    border-radius: 8px;
    font-size: 1.05rem;
    font-weight: 600;
    cursor: pointer;
    transition: transform 0.15s, box-shadow 0.15s;
    margin-top: 8px;
}

.btn-calculate:hover {
    transform: translateY(-2px);
    box-shadow: 0 6px 20px rgba(102, 126, 234, 0.4);
}

```

```

}

.btn-calculate:active {
  transform: translateY(0);
}

.btn-calculate:disabled {
  opacity: 0.5;
  cursor: not-allowed;
  transform: none;
  box-shadow: none;
}

/* ===== Results Panel ===== */
.results-panel {
  margin-top: 24px;
  background: #f8f9ff;
  border-radius: 12px;
  padding: 24px;
  animation: fadeIn 0.4s ease;
}

.results-panel h3 {
  font-size: 1rem;
  color: #667eea;
  margin-bottom: 16px;
  text-align: center;
}

.result-row {
  display: flex;
  justify-content: space-between;
  align-items: center;
  padding: 12px 0;
  border-bottom: 1px solid #e8eaf6;
}

.result-row:last-child {
  border-bottom: none;
}

.result-label {
  font-weight: 500;
  color: #555;
}

.result-value {
  font-size: 1.2rem;
  font-weight: 700;
}

```

```

        color: #302b63;
    }

    /* ===== Error Message ===== */
    .error-message {
        background: #fff0f0;
        color: #d32f2f;
        padding: 12px 16px;
        border-radius: 8px;
        margin-top: 16px;
        font-size: 0.9rem;
        animation: fadeIn 0.3s ease;
    }

    /* ===== Animations ===== */
    @keyframes fadeIn {
        from { opacity: 0; transform: translateY(8px); }
        to   { opacity: 1; transform: translateY(0); }
    }

    /* ===== Responsive ===== */
    @media (max-width: 480px) {
        .property-input {
            flex-direction: column;
            align-items: flex-start;
            gap: 4px;
        }
        .property-input label {
            min-width: auto;
        }
    }
}

```

Step 7: Update the Layout Background

To match the dark gradient background from the Web API version, open Components/Layout/MainLayout.razor and replace its content with:

```
@inherits LayoutComponentBase
```

```

<main>
    @Body
</main>

```

Then create (or open) Components/Layout/MainLayout.razor.css and set:

```

main {
    font-family: 'Segoe UI', Tahoma, Geneva, Verdana, sans-serif;
    background: linear-gradient(135deg, #0f0c29, #302b63, #24243e);
    min-height: 100vh;
    display: flex;
}

```

```
justify-content: center;
align-items: flex-start;
padding: 40px 20px;
color: #333;
}
```

Why separate CSS files? Blazor CSS isolation scopes styles to each component automatically. The compiler appends a unique attribute (e.g., b-abc123) to the component's HTML elements, so styles in `Home.razor.css` only affect `Home.razor` — no class name collisions.

Part 7: Clean Up (Optional)

The default Blazor template includes a navigation sidebar and some sample pages you don't need.

7a. Remove the NavMenu

Open `Components/Layout/NavMenu.razor` and replace its entire content with an empty file, or simply delete all the `<NavLink>` entries.

7b. Remove sample pages

Delete these files (they're from the template):

- `Components/Pages/Counter.razor`
 - `Components/Pages/Weather.razor`
-

Part 8: Run the Application

Step 8: Launch and Verify

1. Press **F5** in Visual Studio to run the project
2. The browser opens directly to the Geometry Calculator — no need to navigate away from Swagger
3. The application is ready to use immediately

Step 9: Test the Workflow

1. **Select a shape** from the dropdown (e.g., Circle)
 2. The relevant **property inputs** appear dynamically (e.g., Radius)
 3. **Enter values** (e.g., Radius = 5)
 4. Click **Calculate**
 5. The **Area** (78.5398 sq. units) and **Circumference** (31.4159 units) appear below
-

Project Structure (Final)

GeometryCalculatorBlazor/		
├── Components/		
│ ├── Layout/		
│ │ ├── MainLayout.razor	← App layout (background styling)	
│ │ └── MainLayout.razor.css	← Layout scoped styles	
│ ├── Pages/		
│ │ ├── Home.razor	← Calculator UI (Razor component)	
│ │ └── Home.razor.css	← Calculator scoped styles	
│ ├── _Imports.razor	← Global using directives	
│ ├── App.razor	← Root component	
│ └── Routes.razor	← Routing configuration	
├── Models/		
│ ├── ShapeRequest.cs	← Input model	
│ └── ShapeResult.cs	← Output model	
├── Services/		
│ ├── IGeometryService.cs	← Service interface	
│ └── GeometryService.cs	← Calculation logic	
├── Program.cs	← App configuration & DI	
└── GeometryCalculatorBlazor.csproj		

Key Differences: Web API vs Blazor Web App

Aspect	Web API + HTML/JS	Blazor Web App
Frontend	Plain HTML/CSS/JavaScript	Razor Components (.razor files)
Communication	HTTP requests (fetch()) to API endpoints	Direct C# method calls via SignalR
Controllers	GeometryController.cs with 3 endpoints	Not needed — service is injected directly
Swagger	Yes (for API testing)	Not needed — no API to document
CORS	Required (separate frontend/backend origins)	Not needed — single application
JavaScript	~180 lines of DOM manipulation	Zero JavaScript — all C#
Styling	Single <style> block in index.html	CSS isolation (.razor.css files)
State management	DOM + JS variables	C# fields in @code block
Deployment	Could be separate frontend/backend	Single deployable application

Key Design Decisions Explained

Why inject the service directly? In Blazor Server, the UI runs on the server. The Razor component and the GeometryService are in the same process — there's no need for HTTP. We register the service with `AddScoped` and use `@inject` to get it, exactly like constructor injection in a controller but without the HTTP overhead.

Why `@oninput` instead of `@bind`? We use `@oninput` with a manual handler because the property values are stored in a `Dictionary<string, double>` with dynamic keys. Blazor's `@bind` works best with known properties. The `@oninput` approach gives us full control over parsing and validation.

Why CSS isolation? Scoped styles prevent naming conflicts and keep each component self-contained. If you later add more pages (e.g., a unit converter), their styles won't interfere with the calculator.

Why no `wwwroot/index.html`? In Blazor Web App (.NET 8), the host page is `App.razor` — it renders the `<html>`, `<head>`, and `<body>` tags using Razor syntax. Static files still live in `wwwroot` (for images, custom CSS, etc.), but the HTML shell is part of the Blazor component tree.

Optional Enhancements

To extend this project, consider adding: unit tests with `xUnit` or `bUnit` for the service and component, input validation using `EditForm` with `DataAnnotations`, a shape diagram rendered as an SVG component that updates dynamically, additional shapes such as `Pentagon` or `Hexagon`, a calculation history panel using a scoped service or `ProtectedLocalStorage`, or deploying to Azure App Service as a single application with no separate frontend/backend to manage.